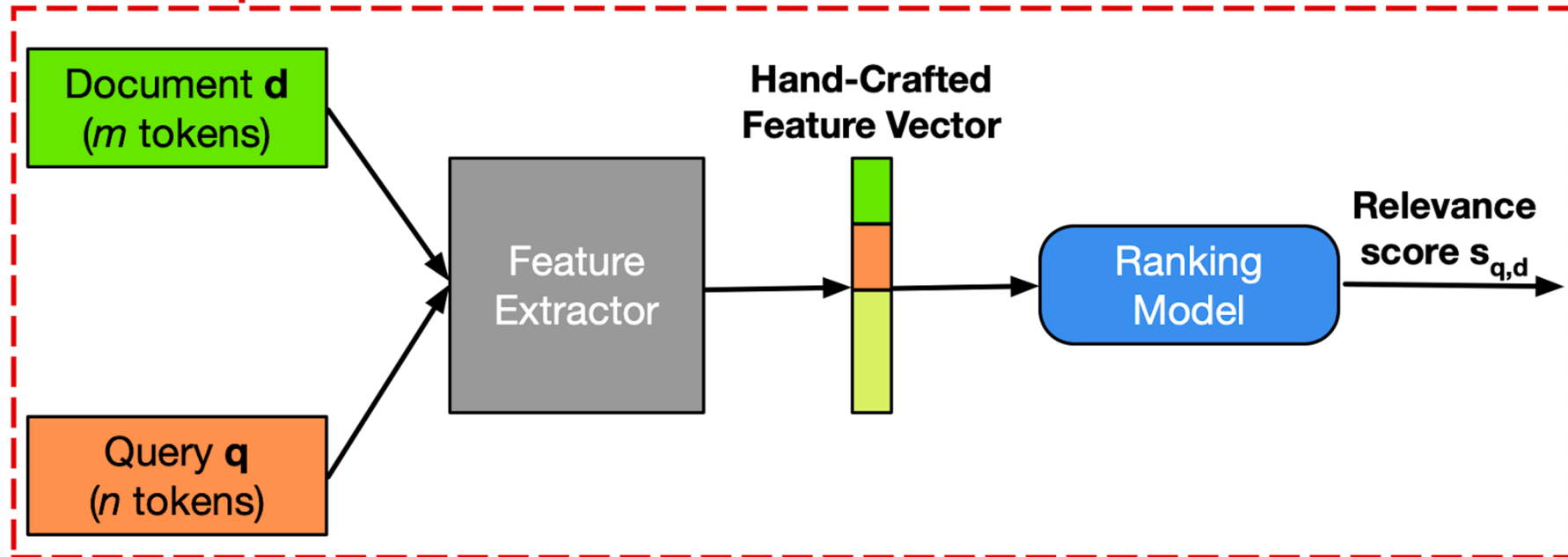


Machine Learning for IR: Learning to Rank

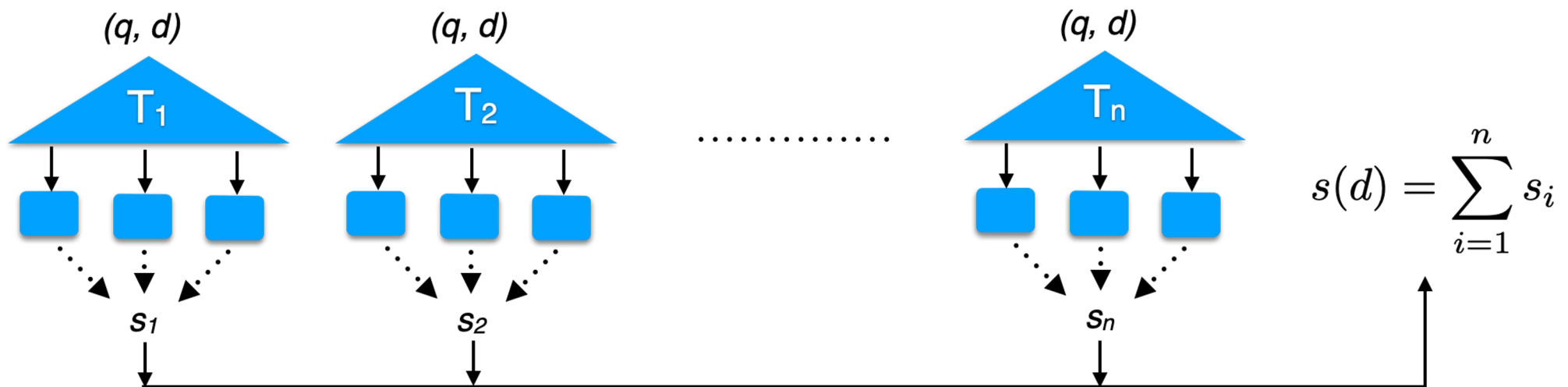
Online Computation



- Input: **vector of features** modeling relevance for query/doc + **labels**
- State-of-the-art approach: forest of **thousands** of regression trees
- **High quality**, but their use is **computationally expensive**!

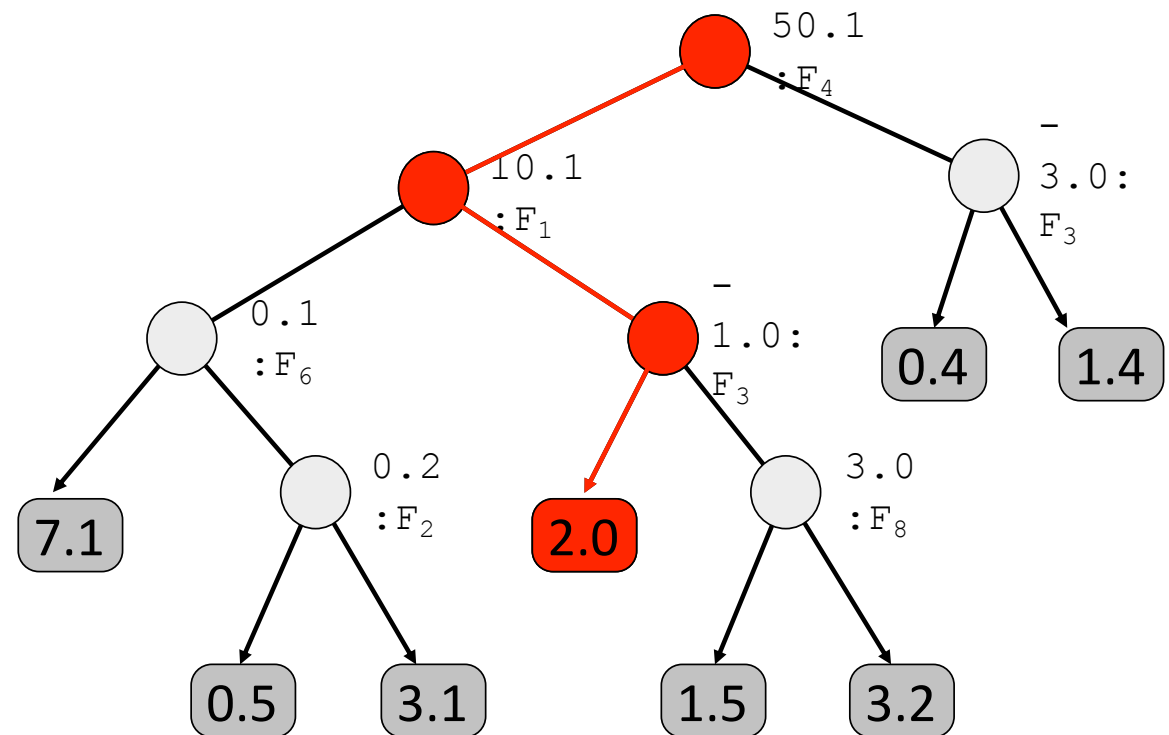
Efficient Machine Learning for IR

- Machine-learned models for precision-oriented ranking
 - **Tree forests:** GBRT, LambdaMART, Random Forest, Oblivious Trees, etc.
 - ensemble of weak learners, each one contributing with a partial score
 - at scoring time, all trees need to be processed independently
- If we assume: 1,000 docs per query, 3,000 trees, a tree depth of 10 nodes
- **Expensive:** 3,000 x 10 (depth) = 30K tests per document. 30K x 1,000 docs = **30M tests per query**

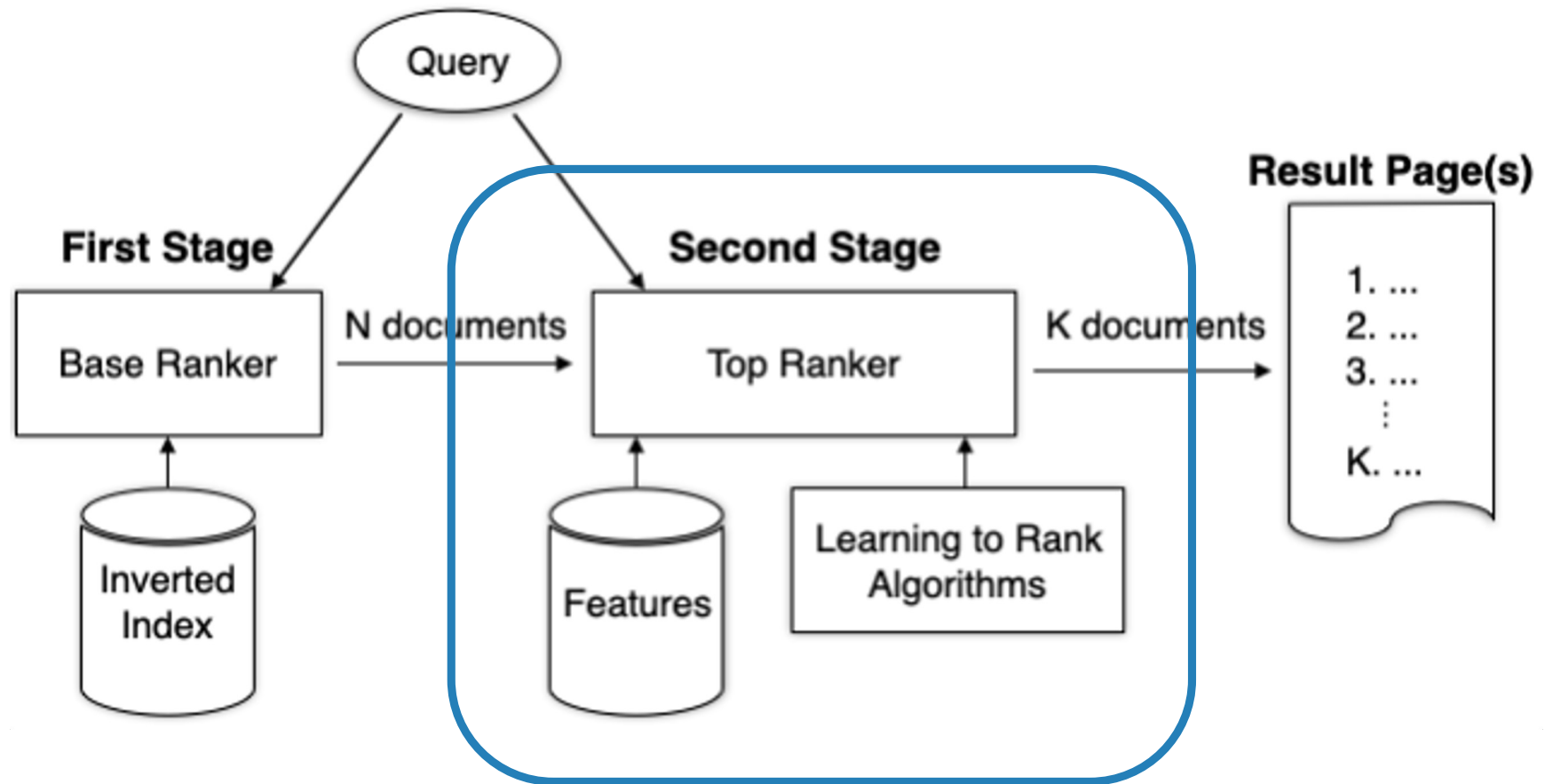


An Example of Tree

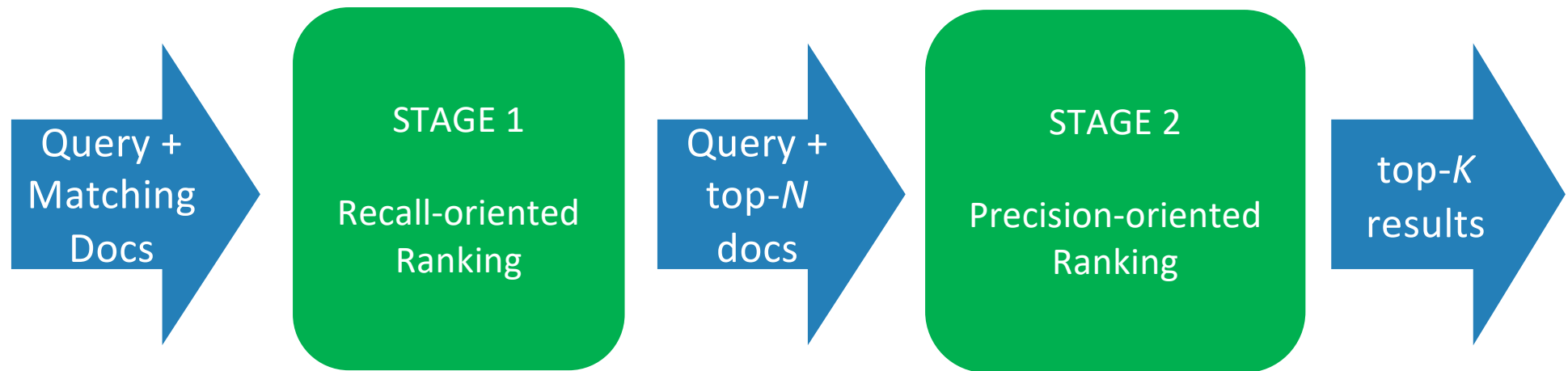
- Internal nodes: Boolean conditions (f, threshold) on features
- Leaves: prediction of relevance
- Relevance of a query-doc pair: sum of all tree predictions
- How are they fitted?



How Do Search Engines Work?

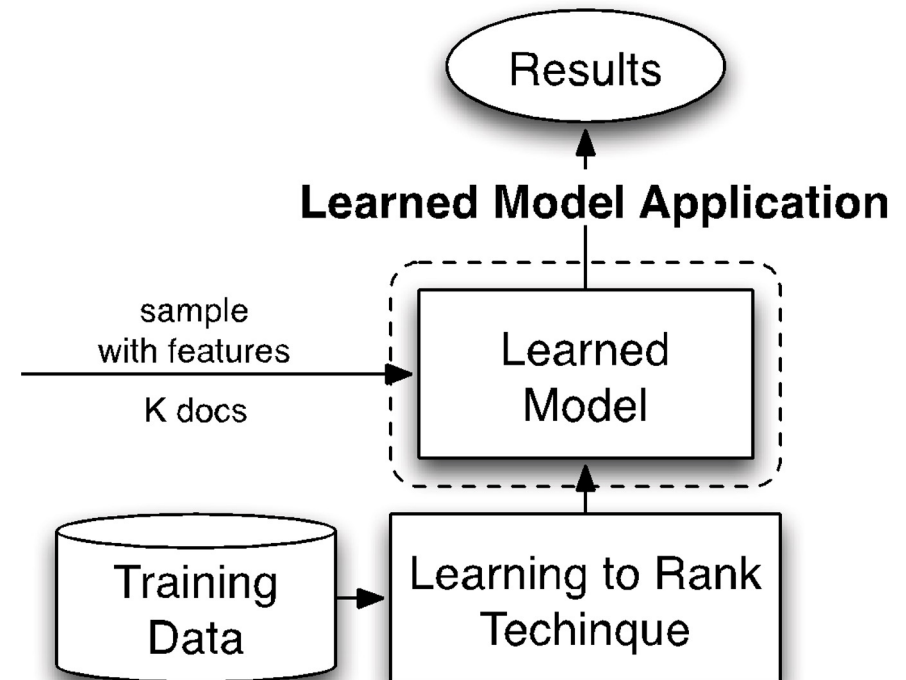


Cascading of Ranking Architectures



Efficiency/Effectiveness Trade-offs

- Efficiency in Learning to Rank (LtR) has been addressed in different ways
- Main research lines
 - Optimizing efficiency within the learning process
 - Approximate score computation and efficient cascades
 - Efficient traversal of tree-based models
- Different impact on the architecture



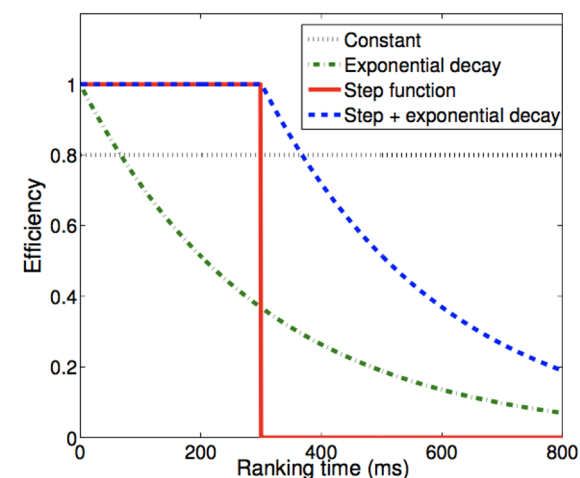
Optimizing Efficiency within the Learning Process

Learning to Efficiently Rank [WLM10]

- Wang *et al.* propose a new cost function for **learning** models that directly optimize the tradeoff metric: Efficiency-Effectiveness Tradeoff Metric (EET)

$$\text{EET}(Q) = \frac{(1 + \beta^2) \cdot (\gamma(Q)) \sigma(Q)}{\beta^2 \cdot \sigma(Q) + \gamma(Q)} \longrightarrow \text{MEET}(R) = \frac{1}{N} \sum \text{EET}(Q)$$

- New efficiency metrics: constant, step, exponential
- Focus on linear feature-based ranking functions
- Learned functions show **significant decreased average query execution times**

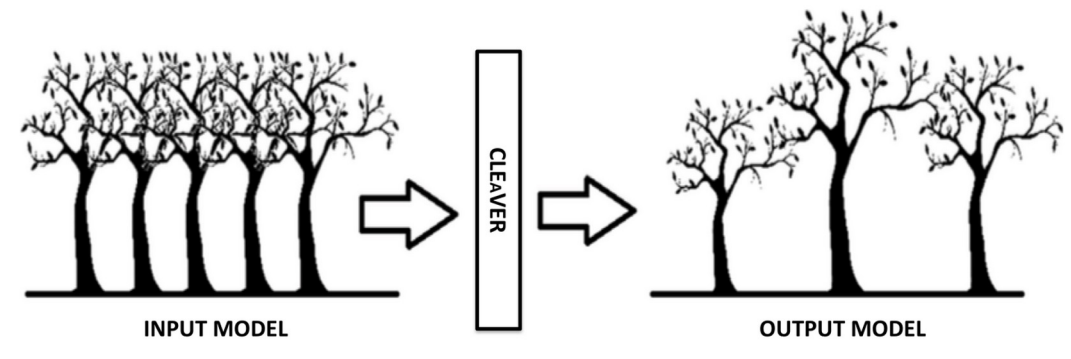


Training Efficient Tree-Based Models for Document Ranking [AL13]

- Asadi and Lin propose techniques for training GBRTs that have efficient runtime characteristics.
 - **compact**, **shallow**, and **balanced** trees yield faster predictions
- Cost-sensitive Tree Induction: jointly minimize the loss and the evaluation cost
- Two strategies
 - By directly modifying the node splitting criterion during tree induction
 - Allow split with maximum gain if it does not increase the maximum depth of the tree
 - Find a node closer to the root which, if split, result in a gain larger than the discounted maximum gain
 - Pruning while boosting with focus on **tree depth** and **density**
 - Additional stages compensate for the loss in effectiveness
 - Collapse terminal nodes until the number of internal nodes reach a balanced tree
- Experiments on MSLR-WEB10K show that the **pruning** approach is superior.
 - 40% decrease in prediction latency with minimal reduction in final NDCG.

CLEAVER [LNO+16a]

- Lucchese et al. propose a **pruning & re-weighting** post-processing methodology
- Several pruning strategies
 - random, last
 - skip, low weights
 - score loss
 - quality loss
- Greedy line search strategy applied to tree weights
- Experiments on MART and LambdaMART
 - MSLR-Web30K and Istella-S LETOR
- Quality loss: **same effectiveness of the original model with up to 20% of the trees**



C. Lucchese, F. M. Nardini, S. Orlando, R. Perego, F. Silvestri, and S. Trani. *Post-learning optimization of tree ensembles for efficient ranking*. In Proc. ACM SIGIR, 2016.

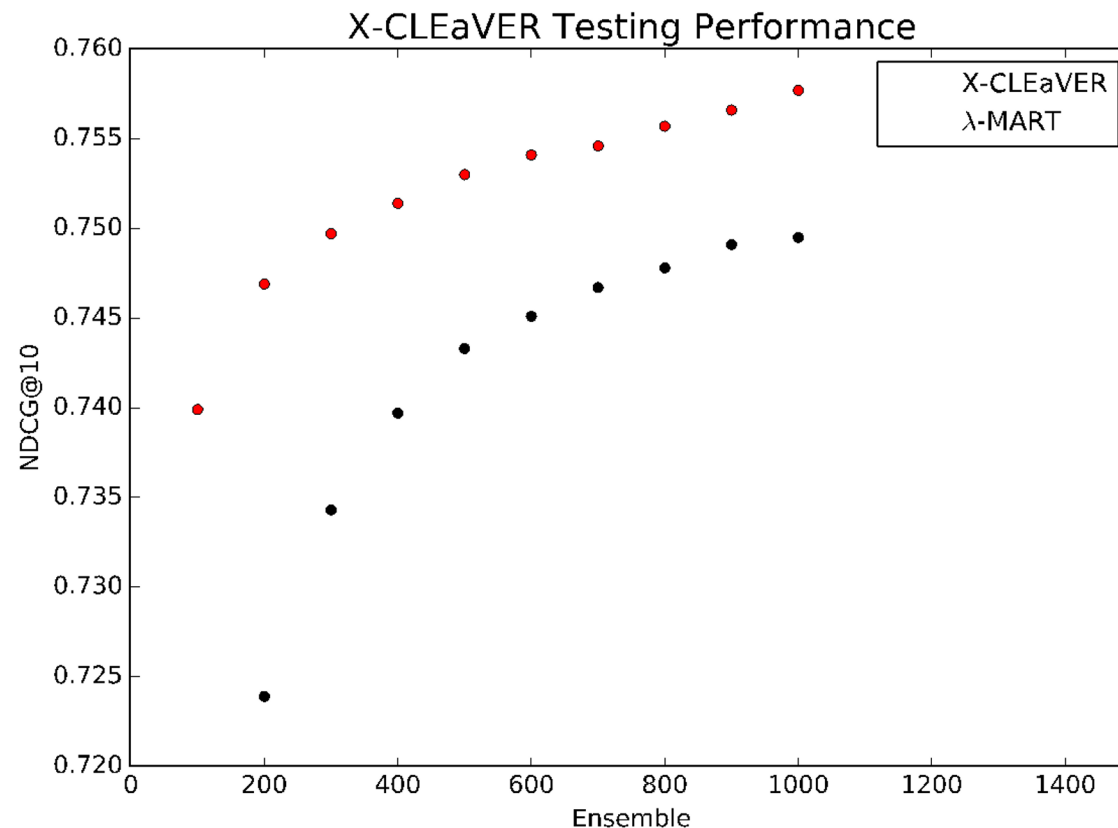
X-CLEAVER [LNO+18]

- Pruning and re-weighting during gradient boosting
 1. Redundant trees are removed from the given ensemble
 2. Weights of the remaining trees are fine-tuned by optimizing the desired ranking quality metric, i.e., NDCG
- Same pruning strategies of CLEAVER
- Experiments on two publicly available datasets: MSN30K-1, Istella-S
 - Pruning and re-weighting during learning are more effective than applying a single post-learning optimization step
 - X-CLEaVER allows to train **even more compact forests with no loss in performance**

[LNO+18] C. Lucchese, F. M. Nardini, S. Orlando, R. Perego, F. Silvestri, and S. Trani. *X-CLEAVER: Learning Ranking Ensembles by Growing and Pruning Trees*. ACM TIST, 2018.

X-CLEAVER [LNO+18]

- Pruning and
 1. Redundant
 2. Weights ranking
- Same pruning
- Experiment:
 - Pruning and single pos
 - X-CLEaVER performance



the desired

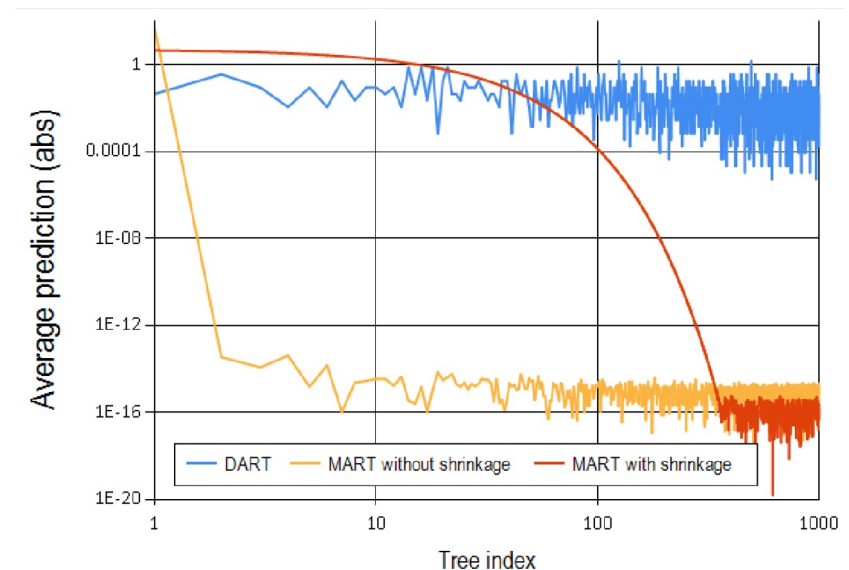
-1, Istella-S
an applying a

loss in

[LNO+18] C. Lucchese, F. M. Nardini, S. Orlando, R. Perego, F. Silvestri, and S. Trani. *X-CLEAVER: Learning Ranking Ensembles by Growing and Pruning Trees*. ACM TIST, 2018.

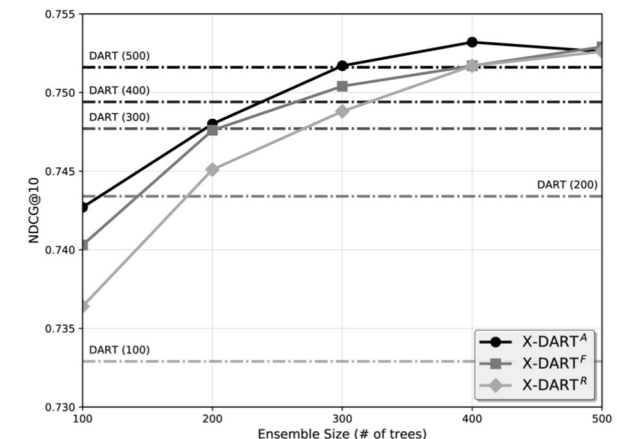
DART [VGB15]

- Rashmi and Gilad-Bachrach propose to employ *dropouts* from NN while learning a MART: DART
 - Dropouts as a way to fight *over-specialization*
 - Shrinkage helps but does not solve
- DART differs from MART
 - When learning a new tree, a subset of the model is muted (random)
 - Normalization step when adding a new tree to avoid *overshooting*
- Experiments on MSLR-Web10K, NDCG@3
 - Improvement over LambdaMART



X-DART [LNO+17]

- Lucchese *et al.* merge DART with pruning while training
 - like DART, some trees are muted and this set is removed after fitting if needed
- Two good news
 - X-DART builds even more compact models than DART
 - Smaller models are less prone to overfitting: potential for higher effectiveness
- Three strategies for pruning trees
 - Ratio, Fixed, Adaptive
- Experiments on MSLR-Web30K and Istella-S
 - X-DART (adaptive) provide statistically significant improvements w.r.t. DART with up to 20% less trees
 - Same effectiveness of DART with up to 40% less trees

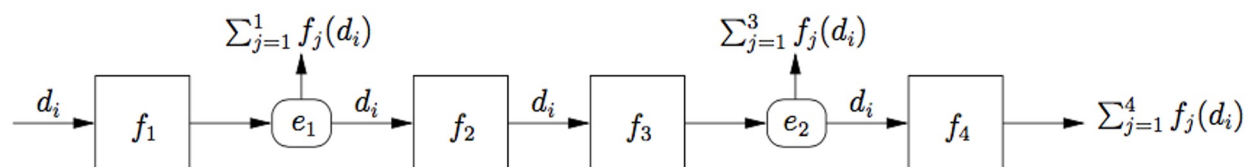


[LNO+17] C. Lucchese, F. M. Nardini, S. Orlando, R. Perego, and S. Trani. *X-DART: Blending dropout and pruning for efficient learning to rank*. In Proc. ACM SIGIR, 2017.

Approximate Score Computation and Efficient Cascades

Early Exit Optimizations for Additive Machine Learned Ranking Systems [CZC+10]

- Why short-circuiting the scoring process in additive ensembles
 - for each query, few highly relevant documents and many irrelevant ones
 - most users view only the first few result pages
- Cambazoglu *et al.* introduce additive ensembles with early exits



- Four techniques
 - early exits using {Score, Capacity, Rank, Proximity} thresholds
- Evaluation on a state-of-the-art ML platform with GBRT
- With EPT, up to four times faster without loss in quality

[CZC+10] B. B. Cambazoglu, H. Zaragoza, O. Chapelle, J. Chen, C. Liao, Z. Zheng, and J. Degenhardt. *Early exit optimizations for additive machine learned ranking systems*. In Proc. ACM WSDM, 2010.